

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №10
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Модульне тестування об'єктно-орієнтованих програм у Python.

Мета роботи: Навчитися створювати модульні тести для класів і функцій у Python з використанням бібліотеки unittest, у тому числі з використанням параметризованих тестів і мок-об'єктів.

Теоретичні відомості

Основи модульного тестування у Python.

Модульне тестування (unit testing) – це підхід у розробці програмного забезпечення, за якого окремі невеликі компоненти програми (функції, методи або класи) тестуються в повній ізоляції від решти системи. Метою є підтвердження того, що кожна окрема одиниця коду працює рівно так, як було задумано. У Python головним інструментом для цього є вбудований модуль unittest.

Тести у unittest організовуються у вигляді класів, що успадковуються від unittest.TestCase. Для перевірки очікуваних результатів використовуються спеціальні методи (asserts), найпопулярнішими з яких є assertEquals, assertTrue, assertFalse та assertRaises (для перевірки правильної генерації помилок, наприклад, при діленні на нуль).

Для оптимізації процесу та уникнення дублювання коду в unittest існують методи життєвого циклу: setUp() викликається перед кожним окремим тестом для підготовки середовища (ініціалізація змінних, об'єктів), а tearDown() – після тесту для очищення пам'яті чи закриття з'єднань.

Просунуті техніки: Параметризація та Мок-об'єкти.

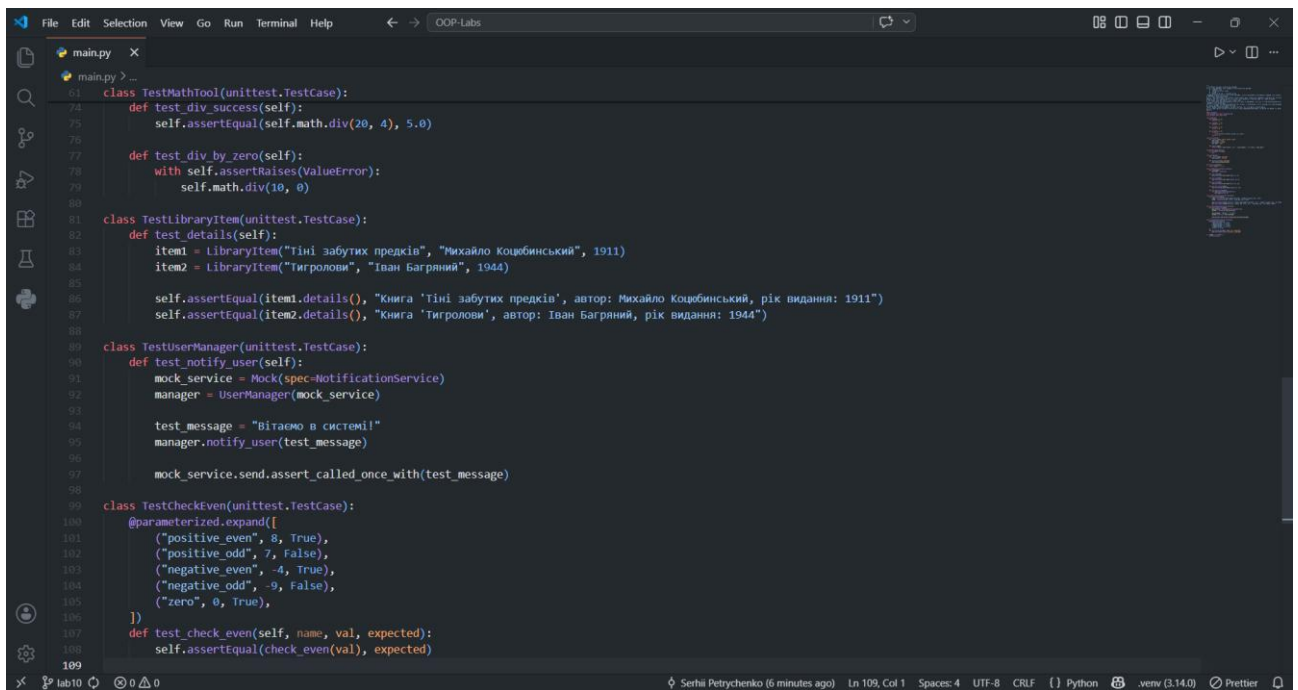
Параметризовані тести: Дозволяють запускати одну й ту саму тестову логіку багаторазово з різними вхідними даними (наприклад, за допомогою бібліотеки parameterized). Це значно зменшує обсяг коду при тестуванні функцій, які мають багато крайових випадків (додатні числа, від'ємні, нуль тощо).

Мок-об'єкти (Mocks): Об'єкти з модуля unittest.mock, які створюються для імітації (підробки) реальних компонентів системи. Вони критично необхідні, коли клас, який тестується, залежить від зовнішніх сервісів (баз даних, мережових API). Мок-об'єкти дозволяють ізолювати логіку класу і перевірити лише факт правильної взаємодії з цими залежностями без їх реального виклику.

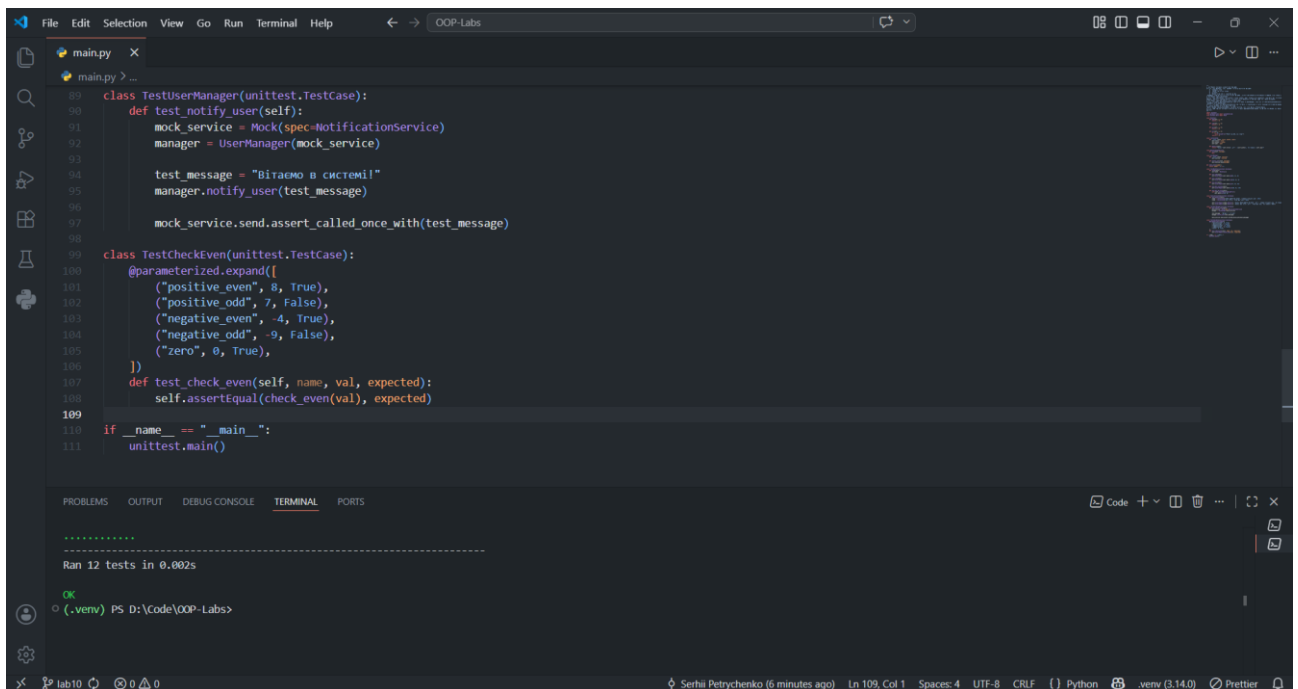
Результати виконаної роботи

```
File Edit Selection View Go Run Terminal Help
main.py X
main.py > MathTool > div
1 """
2 1. Тестування виконання арифметичних операцій
3 Створіть клас MathTool, який реалізує основні арифметичні операції:
4     • складання чисел
5     • різниця між двома числами
6     • множення
7     • ділення з перевіркою на поділ на нуль
8 Реалізуйте модульні тести до кожної з цих операцій. Перевірте поведінку методу ділення у випадку, коли другий аргумент дорівнює нулю.
9
10 2. Тестування класу LibraryItem
11 Створіть клас LibraryItem, який має поля: title, author, year. Додайте метод details(), який повертає рядок із повною інформацією про об'єкт.
12 Напишіть юніт-тести, який перевіряє, чи коректно метод details() формує інформацію для різних прикладів.
13
14 3. Тестування взаємодії класів з використанням mock
15 Створіть два класи: NotificationService (має метод send) та UserManager, який використовує NotificationService для надсилання повідомлень. Класі UserManager
16 реалізуйте метод notify_user.
17 Створіть мок-об'єкт для NotificationService і протестуйте, що notify_user коректно викликає метод send із заданими параметрами.
18
19 4. Параметризовані тести для парності числа
20 Опишіть функцію check_even(number), яка повертає True, якщо число парне, False інакше.
21 Створіть набір параметризованих тестів із використанням @parameterized.expand, щоб перевірити функцію на різних прикладах: позитивні числа, від'ємні, нуль, непарні
22 значення.
23 """
24 import unittest
25 from parameterized import parameterized
26 from unittest.mock import Mock
27
28 class MathTool:
29     def add(self, a, b):
30         return a + b
31
32     def sub(self, a, b):
33         return a - b
34
35     def mul(self, a, b):
36         return a * b
37
38     def div(self, a, b):
39         if b == 0:
40             raise ValueError("Ділити на нуль не можна!")
```

```
File Edit Selection View Go Run Terminal Help
main.py X
main.py > testMathTool > test_mul
36     return a / b
37
38 class LibraryItem:
39     def __init__(self, title, author, year):
40         self.title = title
41         self.author = author
42         self.year = year
43
44     def details(self):
45         return f"Книга '{self.title}', автор: {self.author}, рік видання: {self.year}"
46
47 class NotificationService:
48     def send(self, message):
49         pass
50
51 class UserManager:
52     def __init__(self, service):
53         self.service = service
54
55     def notify_user(self, message):
56         self.service.send(message)
57
58 def check_even(number):
59     return number % 2 == 0
60
61 class TestMathTool(unittest.TestCase):
62     def setUp(self):
63         self.math = MathTool()
64
65     def test_add(self):
66         self.assertEqual(self.math.add(5, 3), 8)
67
68     def test_sub(self):
69         self.assertEqual(self.math.sub(10, 4), 6)
70
71     def test_mul(self):
72         self.assertEqual(self.math.mul(7, 6), 42)
```



```
61 class TestMathTool(unittest.TestCase):
62     def test_div_success(self):
63         self.assertEqual(self.math.div(20, 4), 5.0)
64
65     def test_div_by_zero(self):
66         with self.assertRaises(ValueError):
67             self.math.div(10, 0)
68
69 class TestLibraryItem(unittest.TestCase):
70     def test_details(self):
71         item1 = LibraryItem("Тіні забутих предків", "Михайло Коцюбинський", 1911)
72         item2 = LibraryItem("Тигролови", "Іван Багряний", 1944)
73
74         self.assertEqual(item1.details(), "Книга 'Тіні забутих предків', автор: Михайло Коцюбинський, рік видання: 1911")
75         self.assertEqual(item2.details(), "Книга 'Тигролови', автор: Іван Багряний, рік видання: 1944")
76
77 class TestUserManager(unittest.TestCase):
78     def test_notify_user(self):
79         mock_service = Mock(spec=NotificationService)
80         manager = UserManager(mock_service)
81
82         test_message = "Вітаємо в системі!"
83         manager.notify_user(test_message)
84
85         mock_service.send.assert_called_once_with(test_message)
86
87 class TestCheckEven(unittest.TestCase):
88     @parameterized.expand([
89         ("positive_even", 8, True),
90         ("positive_odd", 7, False),
91         ("negative_even", -4, True),
92         ("negative_odd", -9, False),
93         ("zero", 0, True),
94     ])
95     def test_check_even(self, name, val, expected):
96         self.assertEqual(check_even(val), expected)
97
98 if __name__ == "__main__":
99     unittest.main()
```



```
89 class TestUserManager(unittest.TestCase):
90     def test_notify_user(self):
91         mock_service = Mock(spec=NotificationService)
92         manager = UserManager(mock_service)
93
94         test_message = "Вітаємо в системі!"
95         manager.notify_user(test_message)
96
97         mock_service.send.assert_called_once_with(test_message)
98
99 class TestCheckEven(unittest.TestCase):
100     @parameterized.expand([
101         ("positive_even", 8, True),
102         ("positive_odd", 7, False),
103         ("negative_even", -4, True),
104         ("negative_odd", -9, False),
105         ("zero", 0, True),
106     ])
107     def test_check_even(self, name, val, expected):
108         self.assertEqual(check_even(val), expected)
109
110 if __name__ == "__main__":
111     unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Ran 12 tests in 0.002s
OK
(.venv) PS D:\Code\OOP-Labs

Висновки

1. Було розроблено набір модульних тестів для перевірки об'єктно-орієнтованого коду на Python. Реалізовано тестування математичних операцій класу MathTool (з обробкою виключень при діленні на нуль), перевірено форматування рядків у класі LibraryItem. Також було застосовано параметризовані тести для масової перевірки функції визначення парності числа та використано мок-об'єкт (Mock) для безпечного тестування взаємодії класів UserManager та NotificationService без реального надсилення повідомлень.
2. Написання юніт-тестів є невіддільною частиною сучасного циклу розробки ПЗ (особливо в методологіях CI/CD та TDD). Цей підхід використовується для створення систем автоматичної перевірки коду перед його випуском у продакшн. Будь-які зміни в архітектурі,

рефакторинг або додавання нових функцій супроводжуються запуском тестів, що гарантує відсутність регресій (появи нових помилок у вже стабільному коді).

3. Виконання роботи дозволило на практиці засвоїти принципи створення надійного коду через його покриття тестами. Було отримано навички роботи з вбудованим фреймворком unittest, розуміння методів налаштування тестового середовища (setUp) та досвід ізоляції залежностей за допомогою мок-об'єктів. Це дало розуміння того, як писати код, який легко піддається тестуванню.
4. Переваги: Максимально раннє виявлення логічних помилок, що здешевлює їх виправлення. Тести роблять рефакторинг безпечним і часто слугують найкращою "живою" документацією до коду, адже показують, як саме класи мають використовуватися.
Недоліки: Написання якісних тестів вимагає значного часу (іноді стільки ж, скільки й сам код). При неправильному використанні мок-об'єктів тести можуть проходити успішно, ігноруючи реальні проблеми інтеграції між компонентами. Крім того, тестову базу потрібно постійно підтримувати та оновлювати синхронно зі змінами бізнес-логіки.